

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ciencias Matemáticas

ASIGNATURA DE INTELIGENCIA ARTIFICIAL APLICADA

INTELIGENCIA ARTIFICIAL APLICADA

Fundamentos, Métodos y Validación Científica

Compilación Monográfica de Investigaciones por Grupos

Docente Titular: Frida Mereyda Lopez Cordova

LIMA, PERÚ · 2026

Inteligencia Artificial Aplicada: Fundamentos, Métodos y Validación Científica

Compendio Académico de Proyectos y Monografías de Investigación

Comité Editorial y Autoría de Capítulos

Capítulo 12 (Prototipado Rápido con No-Code, Low-Code e IA):

Pablo Alexis Valdivia Alipazaga · Pablo Gerardo Valdivia Alipazaga · Carlos Arnaldo Huamani Riojas · Adriano Collantes Sánchez · Guillermo Valentino Ayora Príncipe

Equipos de Investigación Participantes:

Estudiantes de los Grupos 1 al 13 de la Asignatura de Inteligencia Artificial Aplicada.

Información Académica e Institucional

Asignatura: Inteligencia Artificial Aplicada

Docente Titular: Frida Mereyda Lopez Cordova

Facultad de Ciencias Matemáticas, Universidad Nacional Mayor de San Marcos

Ciclo Académico: 2026-II

Lima, Perú · 2026

Obra colectiva universitaria elaborada con fines docentes y de investigación en la Facultad de Ciencias Matemáticas de la Universidad Nacional Mayor de San Marcos.

Derechos reservados © 2026. Se autoriza la reproducción parcial o total para fines académicos sin fines de lucro, citando debidamente la fuente y los autores de cada capítulo.

Contenidos

Datos de la Obra I

Presentación 1

12 Prototipado Rápido con LCNC e IA Generativa 2

12.1 Fundamentos, Abstracción Visual y Gobernanza Documental en LCNC 3
12.1.1 Evolución Metodológica del Desarrollo de Software y Abstracción Visual 3
12.1.2 Arquitecturas de Automatización Empresarial y Gobernanza Documental 4
12.1.3 El Ecosistema Power Platform y las Normas de Gobernanza Documental 4
12.1.4 Caso o Aplicación: Automatización del Flujo Documental MAF en la Gestión Académico-Empresarial 5
12.1.5 Demostración y Evaluación Comparativa: Power Platform frente a Scripts Tradicionales en Python 6

12.2 Arquitectura de Microservicios Automatizada desde Modelos UML (FSMicro-Generator) 7
12.2.1 Anatomía del Marco FSMicroGenerator 7
12.2.2 Enfoque de Plantillas y Pila Tecnológica 7
12.2.3 Pila Tecnológica y Ecosistema DevSecOps 8
12.2.4 Ciclo de Vida del Paquete 8
12.2.5 Figuras y Evidencias: Estudio Experimental Comparativo 8
12.2.6 Evaluación Cuantitativa de Eficiencia y Rendimiento 9
12.2.7 Evaluación Cualitativa de Arquitectura (Escala Likert 1-5, ISO/IEC 25010) 9
12.2.8 Caso Práctico Aplicado: Despliegue de un Microservicio de Gestión Científico 10

12.3 Vibe Coding, Inteligencia Artificial Generativa y Gobernanza del Código Sintético 11
12.3.1 Análisis Comparativo de Rendimiento: Desarrollo Manual vs. Low-Code vs. Vibe Coding con LLMs 11
12.3.2 Patologías del Código Generado por IA: Cuello de Botella de Verificación, Arquitecturas de Error y Deuda Técnica 12
12.3.3 Caso Práctico Aplicado: Auditoría de Seguridad y Refactorización . . . 12

12.4 La Escalera de Capacidades y la Supervisión Humana en la Era de la IA 14
12.4.1 Los Cinco Niveles de la Escalera de Capacidades 14
12.4.2 Capa Transversal de Aseguramiento y Evaluación 15
12.4.3 Experiencia Educativa Documentada y Evidencia Experimental Complementaria 15

12.5 Síntesis Metodológica, Matriz de Decisión y Hoja de Ruta para Laboratorios de I+D. 17
12.5.1 Matriz Multidimensional Unificada de Selección Tecnológica 17

12.5.2 Análisis de Stakeholders (Matriz Poder-Interés) y Evaluación de TCO . 18
12.5.3 Evaluación del Costo Total de Propiedad (TCO) 18
12.5.4 Algoritmo Formulado y Reglas de Decisión Arquitectónica 19
12.5.5 Mitigación del Triángulo de Riesgos y Capa de Aseguramiento 19
12.5.6 Plan Maestro e Implementación en Laboratorios de I+D 20

Sobre Esta Obra. 27

Presentación

La convergencia acelerada entre modelos fundacionales de Inteligencia Artificial Generativa, plataformas de abstracción visual de bajo código y arquitecturas de microservicios distribuidos está transformando radicalmente la metodología de investigación y desarrollo tecnológico en la educación superior. La validación empírica de hipótesis científicas, tradicionalmente ralentizada por prolongados ciclos de codificación manual y cuellos de botella infraestructurales, encuentra hoy un nuevo paradigma en el **prototipado científico acelerado** y la **reasignación estratégica de tareas** (*task reallocation*).

Esta obra colectiva, producida por los equipos de investigación de la asignatura de Inteligencia Artificial Aplicada de la Facultad de Ciencias Matemáticas de la Universidad Nacional Mayor de San Marcos (UNMSM), reúne una serie de capítulos monográficos rigurosos orientados a resolver los principales desafíos técnicos, metodológicos y éticos de la inteligencia artificial contemporánea: desde los sesgos cognitivos y alucinaciones en modelos de lenguaje, la alfabetización digital avanzada y la propiedad intelectual, hasta la ingeniería de software guiada por modelos, el *Vibe Coding* y la supervisión responsable mediante la Escalera de Capacidades.

Cada capítulo ha sido diseñado bajo una estructura pedagógica modular uniforme (Apertura con Pregunta Central y Concepto Clave, Desarrollo Conceptual Sustentado, Evidencias Visuales y Métricas, Caso Práctico Aplicado en la tríada Contexto-Análisis-Interpretación, Notas Históricas, Resumen con cuatro pilares estructurados, Conclusiones en tres ejes y Referencias en normativa APA 7ł edición).

Con este esfuerzo, la Universidad Nacional Mayor de San Marcos reafirma su compromiso histórico con la excelencia académica, la innovación tecnológica de vanguardia y la rigurosidad científica al servicio del desarrollo nacional e internacional.

Comité Editorial y Docencia
Inteligencia Artificial Aplicada, Facultad de Ciencias Matemáticas
Universidad Nacional Mayor de San Marcos
Lima, Perú · 2026

CAPÍTULO 12

Creación de Prototipos Rápidos: Herramientas No-Code, Low-Code e Inteligencia Artificial Generativa para Validar Ideas Científicas

Pregunta Central del Capítulo: ¿Cómo converge la orquestación híbrida de plataformas Low-Code deterministas, generadores basados en modelos UML y Vibe Coding asistido por IA generativa para acelerar la validación de hipótesis científicas, garantizando al mismo tiempo gobernanza documental, seguridad y mitigación de deuda técnica?

Objetivo de Aprendizaje: El lector comprenderá la evolución de la abstracción en ingeniería de software, analizará la generación automatizada de microservicios con *FS-MicroGenerator*, evaluará las patologías del código sintético y el marco de gobernanza VIBE4M, y aplicará la Escalera de Capacidades (L0–L4) junto con matrices de decisión para estructurar un plan maestro de prototipado acelerado en laboratorios de investigación.

CONCEPTO CLAVE

Orquestación Híbrida y Reasignación de Tareas

La orquestación híbrida en prototipado científico consiste en integrar deliberadamente la herramienta computacional óptima según el componente del sistema: bajo código determinista para flujos documentales y gobernanza; generación guiada por UML para arquitecturas de backend y microservicios; y Vibe Coding con IA generativa para maquetación ágil e interfaces de usuario. Este paradigma materializa la reasignación estratégica de la carga cognitiva (*task reallocation*), donde la automatización asume la codificación sintáctica repetitiva mientras el investigador lidera la arquitectura y la verificación de calidad.

12.1 Fundamentos, Abstracción Visual y Gobernanza Documental en LCNC

Evolución Metodológica del Desarrollo de Software y Abstracción Visual

La historia de la ingeniería de software está marcada por una búsqueda continua de niveles superiores de abstracción. Como explican Sido y Emon (2024), las metodologías tradicionales orientadas a procesos rígidos —como el Modelo en Cascada (*Waterfall*), el Modelo en Espiral de Boehm y el Modelo en V— estaban diseñadas para entornos con requerimientos estáticos y predecibles. Aunque estos enfoques tradicionales proporcionaban un control de gestión detallado y una documentación exhaustiva, generaban ciclos de desarrollo sumamente estructurados.

La transición hacia metodologías ágiles (como Scrum y Extreme Programming) y el Proceso Unificado Rational (RUP) mitigó parcialmente estos cuellos de botella al introducir la iteración incremental y la adaptación continua. Sin embargo, como señalan Khalfaoui, Khalfaoui y Azmani (2025), el desarrollo de aplicaciones web full-stack modernas sigue enfrentando una sobrecarga cognitiva considerable. Estudios de productividad en ingeniería de software revelan que los desarrolladores dedican aproximadamente el 80 % de su tiempo a tareas de depuración y corrección de errores, de los cuales entre el 40 % y el 50 % corresponde a retrabajo evitable derivado de la codificación manual repetitiva de sintaxis *boilerplate*.

Para superar la sobrecarga de la codificación textual directa, surgieron las plataformas de desarrollo de bajo código y sin código (Low-Code/No-Code Development Platforms, LCDPs/NCDPs). Estas plataformas operan bajo el principio del Diseño Guiado por Modelos (Model-Driven Design, MDD), en el cual la arquitectura, la lógica de negocio y las interfaces de usuario se definen mediante modelos conceptuales gráficos de alto nivel. El motor de la plataforma traduce automáticamente estas especificaciones visuales en código fuente ejecutable o configuraciones interpretadas en tiempo de ejecución.

Como destacan Marian, Neferu y Mitrea (2026) y Sido y Emon (2024), el paradigma low-code democratiza el desarrollo de software al proporcionar interfaces visuales, componentes prediseñados y abstracciones de alto nivel que permiten a los "desarrolladores ciudadanos" e investigadores participar activamente en la construcción de aplicaciones funcionales sin requerir una formación técnica profunda en sintaxis de programación.

Sido y Emon (2024) destacan que la distinción fundamental entre el desarrollo Low-Code y No-Code radica en la audiencia objetivo y el nivel de extensibilidad técnica:

- **Plataformas No-Code:** Orientadas a usuarios de negocio y científicos sin conocimientos de programación. Utilizan interfaces puramente visuales de arrastrar y soltar (*drag-and-drop*) con bloques funcionales cerrados. Son ideales para prototipos de complejidad baja a moderada, formularios de recolección de datos y automatizaciones de flujos simples.
- **Plataformas Low-Code:** Diseñadas para coordinar el trabajo entre desarrolladores ciudadanos y profesionales informáticos. Aunque ofrecen abstracción visual para el 80 % de la aplicación, permiten la inyección de código personalizado (JavaScript, Python, C#, SQL) para algoritmos complejos e integraciones personalizadas.

Arquitecturas de Automatización Empresarial y Gobernanza Documental

En el ámbito científico y académico, el prototipado rápido no solo requiere la generación ágil de interfaces, sino también un estricto control de la gobernanza documental, la trazabilidad de decisiones y la auditoría de procesos. La investigación desarrollada por Marian, Neferu y Mitrea (2026) demuestra cómo las plataformas Low-Code empresariales integradas en la nube —específicamente el ecosistema Microsoft Power Platform— proporcionan un entorno robusto para operacionalizar flujos de trabajo documentales complejos bajo estándares de cumplimiento industrial.

A diferencia de los entornos de codificación tradicional y del *vibe coding* asistido puramente por inteligencia artificial estocástica, las plataformas low-code ofrecen un marco estructurado que equilibra accesibilidad y rigurosidad técnica. El análisis sistemático formulado por Marian, Neferu y Mitrea (2026) y Sido y Emon (2024) evidencia que las plataformas de bajo código mantienen un control arquitectónico estructurado, dado que el diseño de las soluciones se encuentra delimitado y gobernado por la arquitectura y los esquemas deterministas de la plataforma proveedora.

Asimismo, Marian, Neferu y Mitrea (2026) y Sido y Emon (2024) subrayan que el desarrollo low-code reduce sustancialmente los riesgos de seguridad en comparación con el código generado de manera probabilística o manual sin auditorías exhaustivas. Al heredar protocolos nativos de gobernanza, autenticación empresarial y capas de validación preconfiguradas, se minimiza la superficie de vulnerabilidades ocultas y la acumulación acelerada de deuda técnica en los entornos experimentales de I+D.

La arquitectura de una solución Low-Code empresarial se organiza conceptualmente en tres capas interconectadas:

1. **Capa de Interfaz de Usuario (UI):** Desarrollada mediante aplicaciones de lienzo o basadas en modelos (Power Apps), ofreciendo formularios interactivos con validación de datos en tiempo real y paneles de control (dashboards) adaptados al rol del usuario.
2. **Capa de Lógica de Negocio y Orquestación:** Ejecutada por motores de flujos de trabajo (Power Automate), encargados de gestionar los estados de aprobación, las transiciones departamentales, el envío automatizado de notificaciones y la generación sintética de documentos finales.
3. **Capa de Almacenamiento de Datos y Persistencia:** Diseñada mediante repositorios estructurados (SharePoint Online / Microsoft Dataverse), respaldada por autenticación centralizada y control de acceso basado en roles (RBAC) mediante Microsoft Entra ID.

El Ecosistema Power Platform y las Normas de Gobernanza Documental

Un aspecto crítico resaltado por Marian, Neferu y Mitrea (2026) es la capacidad de las soluciones de bajo código para garantizar el cumplimiento de normas internacionales de gestión de registros e información. A diferencia de los prototipos construidos *ad hoc* con scripts individuales de Python o PHP, las soluciones Low-Code integradas en ecosistemas de nube empresarial heredan automáticamente controles de gobernanza como:

- **Sellado de Tiempo Digital e Identificadores Únicos:** Cada decisión de aprobación o rechazo en el flujo de trabajo queda registrada de forma inmutable con una marca de tiempo precisa y un identificador de auditoría.
- **Control de Versiones Automático:** Las modificaciones e iteraciones sobre documentos científicos se gestionan de forma nativa en bibliotecas con control de versiones, impidiendo la pérdida de historial de revisión.
- **Control de Acceso Centralizado:** La autenticación exige verificación multifactorial y aplica directivas organizacionales para clasificar la sensibilidad de los datos e impedir la pérdida de activos de investigación.

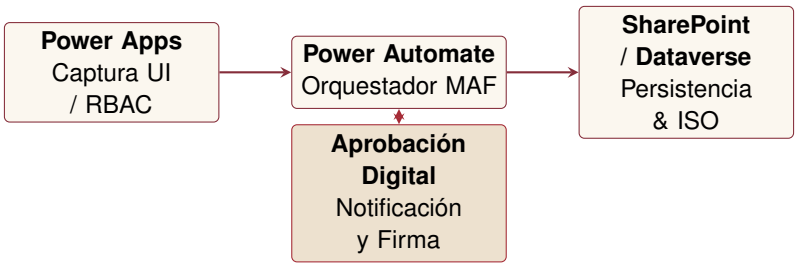
Caso o Aplicación: Automatización del Flujo Documental MAF en la Gestión Académico-Empresarial

Para ilustrar la aplicación práctica de estos conceptos, analicemos el caso de la automatización del proceso de aprobación de solicitudes documentales complejas (*Multiple Attachments Flow*, MAF), documentado por Marian, Neferu y Mitrea (2026). En entornos universitarios y centros de investigación, la aprobación de protocolos de investigación, solicitudes de subvención (*grants*) y autorizaciones de compras de equipamiento científico suele sufrir retrasos debido al intercambio ineficiente de correos electrónicos y formularios impresos.

Descripción del Caso y Resultados Empíricos

El sistema desarrollado integra Power Apps como interfaz de captura estructurada, SharePoint Online como base de datos ligera utilizando columnas de búsqueda (*Lookup Columns*) para simular relaciones relacionales sin el costo de licencias de bases de datos dedicadas, y Power Automate para la orquestación.

Figura 1.1. Arquitectura del Flujo Documental Multitarea (MAF) en Power Platform



Fuente: Adaptación metodológica basada en Marian, Neferu y Mitrea (2026).

En un estudio piloto controlado de seis semanas con 25 usuarios activos en un entorno académico-industrial, se obtuvieron los siguientes resultados:

- **Reducción del Tiempo de Ciclo:** El tiempo medio de aprobación de solicitudes documentales se redujo en un 65.6 %, pasando de 3.2 días en el proceso manual tradicional a solo 1.1 días con la solución Low-Code automatizada.

- **Usabilidad y Adopción:** El 92 % de los participantes calificó la interfaz de la aplicación como "intuitiva" o "muy intuitiva", y el 88 % completó su primera presentación documental sin requerir asistencia técnica.

Demostración y Evaluación Comparativa: Power Platform frente a Scripts Tradicionales en Python

Para comprobar empíricamente las capacidades de automatización e iteración de datos, se contrastó el desempeño de una arquitectura low-code basada en Microsoft Power Platform (Power Apps y Power Automate) frente a la implementación tradicional mediante scripts manuales en Python (utilizando librerías como ReportLab, Pandas y smtplib).

Los resultados empíricos derivados de una evaluación controlada con 25 participantes revelaron las siguientes ventajas determinantes a favor de la plataforma de bajo código:

- **Reducción del tiempo de desarrollo e implementación:** La construcción y puesta en marcha del flujo automatizado en Power Automate demandó un promedio de $\sim$2 horas, en marcado contraste con las $\sim$6 horas requeridas para codificar, depurar y configurar el script en Python.
- **Disminución de la carga de pruebas y depuración:** Power Platform redujo el tiempo total de desarrollo y pruebas en un 80 % a 90 % en comparación con el enfoque tradicional. Mientras que la depuración en Python exigió el rastreo manual de excepciones y líneas de código, Power Automate proporcionó historiales de ejecución detallados por acción y control granular de fallos mediante bloques condicionales visuales (*Configure Run After*).
- **Superioridad en iteración rápida de flujos de datos:** Power Platform superó con holgura a los scripts tradicionales en Python al simplificar la iteración rápida sobre flujos de trabajo de datos, permitiendo reconfigurar reglas lógicas y estructuras documentales desde la interfaz gráfica sin necesidad de refactorizar ni redesplegar código fuente.
- **Seguridad y gobernanza integrada:** El flujo en Power Platform gestionó las credenciales y el cifrado de comunicaciones directamente bajo Azure Active Directory / Microsoft Entra ID y directivas de cumplimiento, superando la gestión manual de credenciales inherente a los scripts de automatización convencionales.

12.2 Arquitectura de Microservicios Automatizada desde Modelos UML (FSMicroGenerator)

El desarrollo acelerado de aplicaciones web modernas para la investigación científica enfrenta barreras severas debido al incremento en la complejidad arquitectónica y la necesidad de reducir el tiempo de salida al mercado (*time-to-market*).

En el contexto de la validación de hipótesis científicas, la mayoría de prototipos iniciales se construyen mediante **arquitecturas monolíticas**.

1. **Falta de aislamiento de fallos:** Un error en un módulo secundario puede comprometer la disponibilidad de todo el sistema distribuido.
2. **Imposibilidad de escalado granular:** No es posible asignar recursos computacionales exclusivamente a los módulos con mayor demanda de procesamiento sin duplicar la aplicación completa.
3. **Alta barrera de entrada para microservicios:** Aunque la arquitectura de microservicios soluciona estos problemas al dividir la aplicación en servicios independientes y desacoplados, introduce una alta complejidad técnica en orquestación, comunicación interservicio y configuración de infraestructura, sobrecargando a equipos de investigación con personal limitado.

Anatomía del Marco FSMicroGenerator

CONCEPTO CLAVE

Descomposición por Dominios (DDD)

FSMicroGenerator aplica los principios de *Domain-Driven Design* (DDD) para descomponer sistemas complejos en contextos delimitados (*bounded contexts*) mediante una jerarquía de tres niveles: **Paquete** (unidad funcional básica), **Módulo** (colaboración de paquetes) y **Proyecto** (sistema distribuido listo para producción).

Enfoque de Plantillas y Pila Tecnológica

El motor de generación opera mediante la articulación de dos tipos de plantillas:

- **Plantillas Estáticas:** Definen el esqueleto arquitectónico estándar del proyecto, incluyendo la configuración del *API Gateway*, utilidades compartidas, estructura base del *frontend* y la integración de microservicios transversales (gestión de usuarios, autenticación y taxonomía).

- **Plantillas Dinámicas:** Transforman los diagramas de clases UML definidos por el usuario en artefactos específicos: esquemas Mongoose, controladores, repositorios, rutas Express.js, pruebas unitarias/integración con Jest, documentación Swagger y componentes *frontend* dinámicos en Angular/PrimeNG.

Pila Tecnológica y Ecosistema DevSecOps

FSMicroGenerator unifica el desarrollo mediante la pila **MEAN** (MongoDB, Express.js, Angular, Node.js), eliminando la fragmentación de lenguajes entre *frontend* y *backend*.

- **Comunicación Interservicio:** *RabbitMQ* para mensajería asíncrona y llamadas RPC síncronas de baja latencia.
- **Caché y Rendimiento:** *Redis* como capa de almacenamiento temporal para reducir la latencia de lectura.
- **Observabilidad Avanzada:** Instrumentación nativa con *Prometheus* (exposición de endpoints `/metrics`) y tableros interactivos en *Grafana*.
- **DevSecOps Shift-Left:** Integración nativa en pipelines de *GitLab CI/CD* con análisis estático de código (SAST), escaneo de dependencias, detección de secretos, escaneo de contenedores con *Trivy* e inspección de infraestructura como código (IaC) con *Checkov*.

Ciclo de Vida del Paquete

El proceso de construcción consta de cuatro etapas principales:

1. **Fase de Creación:** Definición semimanual del modelo conceptual mediante diagramas de clases UML (entidades, atributos, relaciones y restricciones) y vista previa en tiempo real.
2. **Fase de Validación:** Revisión técnica por un usuario calificado para detectar incoherencias en el modelo antes de la ejecución.
3. **Fase de Adaptación:** El motor extrae la configuración, despliega las plantillas estáticas y dinámicas, genera el código fuente y realiza un *push* automático a GitLab.
4. **Fase de Preintegración:** Realización de pruebas CRUD y personalizaciones avanzadas de lógica de negocio por parte del equipo técnico.

Figuras y Evidencias: Estudio Experimental Comparativo

Para validar la efectividad de *FSMicroGenerator*, Khalfaoui et al. (2025) ejecutaron un estudio experimental controlado comparando dos enfoques de desarrollo para un sistema distribuido de gestión de inventarios bajo un límite estricto de **8 horas**:

- **Desarrollo Manual:** 5 desarrolladores experimentados (2-3 años de experiencia en pila MEAN y microservicios).
- **Desarrollo con FSMicroGenerator:** 2 desarrolladores junior (1 y 2 años de experiencia).

CONCEPTO CLAVE

Normalización de Cobertura

Dado que el desarrollo manual no logró completar el sistema en el tiempo límite, se aplicaron los coeficientes de cobertura funcional ($R_f = 0.61$) y no funcional ($R_{nfr} = 0.33$) para proyectar equitativamente los métricos a alcance completo mediante el factor $R_f \times R_{nfr} = 0.2013$.

Evaluación Cuantitativa de Eficiencia y Rendimiento

Métrica de Evaluación	Desarrollo Manual (Bruto)	Desarrollo Manual (Normalizado)	FSMicroGenerator (Junior)	Impacto / Ventaja
Tiempo de Desarrollo	8h (Incompleto) *[35]*	∼40h (Proyectado) *[36]*	**3 horas (100 % completo)** *[35]*	Reducción del 92.5 % de tiempo *[36]*
Cobertura Funcional (R_f)	61 % *[34]*	100 % *[32]*	**100 %** *[34]*	Cobertura funcional total *[34]*
Cobertura No Funcional (R_{nfr})	33 % *[34]*	100 % *[32]*	**100 %** *[34]*	Cobertura NFR nativa *[34]*
Commits en Git	115 commits *[35]*	∼571 commits *[36]*	**11 commits** *[35]*	Estructura estable sin *code churn [35, 38]*
Intervenciones Manuales	1,305 mods. *[35]*	>25,000 mods. *[36]*	**148 mods.** *[35]*	Automatización masiva *[36]*
Rendimiento JMeter	25.44 req/s *[37]*	5.12 req/s *[37]*	**21.75 req/s** *[37]*	+324 % capacidad proc. *[37]*
Tasa de Error	0.0 % *[37]*	0.0 % *[37]*	**0.0 %** *[37, 39]*	Estabilidad operativa *[37]*

Evaluación Cualitativa de Arquitectura (Escala Likert 1-5, ISO/IEC 25010)

Los evaluadores independientes asignaron la máxima calificación (**5/5**) a la solución generada por *FSMicroGenerator* frente a un promedio de (**2/5** y **3/5**) en el desarrollo manual en todas las dimensiones:

1. **Modularidad Arquitectónica (5/5):** Separación perfecta entre *frontend*, *gateway* y microservicios independientes alineados a dominios del negocio.
2. **Reusabilidad de Código (5/5):** Reutilización extensiva mediante componentes compartidos, interceptores y directivas genéricas.
3. **Claridad y Estándares (5/5):** Consistencia total en la convención de nombres, estructura interna MVC y especificación OpenAPI/Swagger.

4. **Mantenibilidad (5/5):** Pruebas unitarias/integración integradas por cada microservicio, reduciendo dramáticamente el riesgo de regresiones.

Caso Práctico Aplicado: Despliegue de un Microservicio de Gestión Científico

---

Contexto

Despliegue del ecosistema **Smart DLSP** (*Smart Digital Logistic Services Providers*), un proyecto de innovación nacional en Marruecos ejecutado por Smart Automation Technologies para modernizar la gestión logística y de carga.

Análisis

Se utilizó *FSMicroGenerator* para modelar la arquitectura completa mediante diagramas de clases UML, generando automáticamente microservicios en Node.js/Express.js, interfaz en Angular y orquestación en Docker Compose.

Interpretación

El uso de *FSMicroGenerator* garantizó la consistencia arquitectónica entre servicios heterogéneos y redujo el tiempo de desarrollo de semanas a días.

12.3 Vibe Coding, Inteligencia Artificial Generativa y Gobernanza del Código Sintético

El avance vertiginoso de los modelos de lenguaje de gran escala (LLMs) especializados en código —tales como OpenAI Codex, Claude 3.5 Sonnet, GitHub Copilot y plataformas dedicadas como Cursor AI y Claude Code— ha dado origen a una transformación radical conocida popularmente como Vibe Coding.

Para formalizar académicamente este fenómeno, Jauhari, Fathullah, Permana y Nugraha (2026) ejecutaron una Revisión Sistemática de la Literatura (Systematic Literature Review, SLR) aplicando rigurosamente los estándares del protocolo PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses). Tras una búsqueda comprehensiva en bases de datos científicas internacionales (IEEE Xplore, ACM Digital Library, ScienceDirect, Scopus y arXiv) y un filtrado metodológico estricto, seleccionaron y analizaron 61 estudios académicos arbitrados publicados entre 2023 y 2026.

Los hallazgos del estudio de Jauhari et al. (2026) confirman que el Vibe Coding representa la interfaz de desarrollo de menor fricción sintáctica en la historia de la informática. El investigador expresa la vibra o intención funcional de la aplicación en lenguaje natural (por ejemplo: "Crea una interfaz en React que capture archivos CSV de datos espectroscópicos, limpie los valores nulos y los grafique en una curva interactiva con D3.js"), y el agente de IA genera instantáneamente el código fuente, la configuración de estilos y los componentes requeridos.

Análisis Comparativo de Rendimiento: Desarrollo Manual vs. Low-Code vs. Vibe Coding con LLMs

A partir de la síntesis cuantitativa y cualitativa de las 61 investigaciones analizadas por Jauhari et al. (2026), se establece un marco comparativo entre los tres paradigmas dominantes de creación de software:

A. **Velocidad y Productividad de Prototipado:** La investigación sistemática demuestra de forma consistente que la asistencia por IA en la generación de código permite reducir el tiempo de desarrollo de un Mínimo Viable de Prueba (MVP) entre un 40 % y un 60 % en comparación con el desarrollo manual convencional.

B. **Nivel de Predictibilidad y Control Arquitectónico:** A diferencia de las plataformas Low-Code (que imponen límites rígidos a través de componentes visuales deterministas que garantizan la consistencia del sistema), el Vibe Coding opera sobre modelos estocásticos probabilísticos. La misma instrucción en lenguaje natural puede producir variaciones en la arquitectura del código generado, exigiendo un mayor nivel de inspección por parte del usuario.

C. Matriz Comparativa Multidimensional de Paradigmas

Dimensión Técnica	Desarrollo Manual Tradicional	Low-Code / No-Code Determinista	Vibe Coding / IA Generativa Pura
Velocidad de Prototipado	Baja a Moderada (Semanas/Meses)	Alta (MVP en Horas/Días)	Extremadamente Alta (Aceleración entre 40 % y 60 %)
Control Arquitectónico	Total y Absoluto (Línea a línea)	Moderado-Alto (Marco visual)	Variable / Probabilístico (Prompt)
Habilidades Requeridas	Sintaxis compleja, algoritmos.	Lógica de negocio, modelado UML.	Ingeniería de prompts, verificación.
Riesgo de Vulnerabilidades	Controlable manualmente.	Bajo (RBAC y gobernanza nativa).	Alto sin SAST (Fallos OWASP).
Flexibilidad de Expresión	Limitada por lenguaje.	Limitada por componentes visuales.	Máxima (Lenguaje natural libre).
Acumulación de Deuda Técnica	Manejable vía refactorización.	Controlada por la plataforma.	Muy Alta si no se aplica VIBE4M.

Patologías del Código Generado por IA: Cuello de Botella de Verificación, Arquitecturas de Error y Deuda Técnica

A pesar de su velocidad, la adopción del Vibe Coding sin controles metodológicos introduce patologías técnicas graves:

1. **El Cuello de Botella de Verificación (Verification Bottleneck):** Como destacan Jauhari et al. (2026), la IA no elimina el esfuerzo de desarrollo, sino que desplaza masivamente la carga cognitiva hacia la fase posterior de pruebas e inspección. Se genera un efecto de congestión donde el desarrollador pasa la mayor parte del tiempo intentando leer, comprender, probar y depurar grandes volúmenes de código sintético que no escribió manualmente.
2. **Arquitecturas de Error y Crisis de Deuda Técnica:** Investigaciones cuantitativas recientes (van Hurne, 2025; Zhao et al., 2025) demuestran que el código producido tiende a duplicar funciones, omitir el manejo de excepciones de borde (edge cases), ignorar principios SOLID y acumular una elevada deuda técnica.
3. **Vulnerabilidades Ocultas de Seguridad:** El estudio de Jauhari et al. (2026) señala que entre el 30 % y el 45 % del código generado sin supervisión por LLMs modernos contiene vulnerabilidades de seguridad clasificadas en el Top 10 de OWASP:
   - Inyección SQL y desinfectación deficiente de entradas de usuario.
   - Uso de claves secretas (hardcoded credentials) o tokens incrustados en texto plano.
   - Librerías desactualizadas con vulnerabilidades conocidas (CVEs).
   - Configuración insegura de políticas CORS (Cross-Origin Resource Sharing).

Caso Práctico Aplicado: Auditoría de Seguridad y Refactorización

Contexto

Un prototipo de Flask generado por IA incluyó vulnerabilidades de Inyección SQL directa:

```
# Código generado por IA (Inseguro - Inyección SQL)
@app.route('/get_data')
def get_data():
    filename = request.args.get('filename')
    query = f"SELECT * FROM measurements WHERE file = '{filename}'"
    result = db.engine.execute(query)
    return jsonify([dict(row) for row in result])
```

Análisis y Refactorización

El marco VIBE4M interceptó el fallo vía Bandit (SAST) en CI/CD. El investigador refactorizó el código usando *safe pathing* y consultas parametrizadas:

```
# Código refactorizado bajo gobernanza VIBE4M (Seguro)
@app.route('/get_data')
def get_data():
    filename = request.args.get('filename')
    if not filename or not re.match(r'^[a-zA-Z0-9_\-\.]+\.csv$', filename):
        return jsonify({'error': 'Nombre de archivo inválido'}), 400
    stmt = text("SELECT * FROM measurements WHERE file = :fn")
    result = db.engine.execute(stmt, {'fn': filename})
    return jsonify([dict(row) for row in result])
```

12.4 La Escalera de Capacidades y la Supervisión Humana en la Era de la IA

Las herramientas de IA pueden acelerar la elaboración de código, documentos y primeras versiones de una solución.

Por ello, la formación debe desarrollar capacidades para definir requisitos, detectar errores, justificar decisiones y comprobar el comportamiento de los sistemas.

Memari y Rudolph (2026, secciones 2–4) fundamentan su propuesta en una revisión narrativa de literatura sobre trabajo, ingeniería de software y educación.

Para una guía de prototipado científico, este enfoque permite formular una distinción metodológica: construir software que funciona y validar una hipótesis científica son actividades relacionadas, pero requieren evidencias diferentes.

Los Cinco Niveles de la Escalera de Capacidades

La Escalera de Capacidades clasifica los sistemas según su autonomía operativa y las responsabilidades humanas necesarias para supervisarlos (Memari y Rudolph, 2026, sección 5).

Nivel	Característica principal	Evidencias para evaluar su funcionamiento
L0. Disparador	Un evento inicia un proceso: envío de un formulario, programación horaria o recepción de un webhook.	Descripción del evento, entradas y explicación de cuándo se activa.
L1. Automatización	Una acción se ejecuta a partir de un disparador.	Regla de automatización, caso de prueba y resultado observado.
L2. Flujo de trabajo	Un proceso fijo encadena varios pasos entre herramientas: recopilar, validar, enviar y registrar información.	Diagrama del flujo, reglas de validación, pruebas y manejo de errores.
L3. Agente de IA	Un sistema orientado a una meta planifica y actúa bajo verificación humana.	Meta, configuración, muestras de resultados, lista de verificación y punto de aprobación humana.
L4. Equipo de agentes	Varios agentes distribuyen tareas y presentan resultados bajo supervisión humana.	Mapa de funciones, estructura de delegación, registro de auditoría, análisis de riesgos e informe de decisión humana.

Fuente de la tabla: adaptación de Memari y Rudolph (2026, figura 1 y tabla 2).

Los niveles superiores requieren mayor responsabilidad de supervisión.

La selección del nivel debe responder al problema y a la capacidad de verificar el sistema.

Capa Transversal de Aseguramiento y Evaluación

El marco incorpora cinco componentes de aseguramiento aplicables en todos los niveles (Memari y Rudolph, 2026, sección 5):

1. **Verificación:** comprobar los resultados mediante pruebas, revisión y detección de errores.
2. **Seguridad:** identificar riesgos, proteger información y controlar las acciones autorizadas.
3. **Documentación:** registrar configuraciones, decisiones, pruebas y uso de herramientas.
4. **Ética:** considerar atribución, privacidad, posibles daños y cumplimiento de las políticas aplicables.
5. **Aprobación humana:** establecer quién revisa y autoriza los resultados o acciones que requieren supervisión.

Estos componentes deben concretarse según las características y los riesgos del sistema.

Divulgación del uso de IA

Memari y Rudolph (2026, sección 7) proponen una plantilla breve que permita responder:

- ¿Qué herramienta de IA se utilizó?
- ¿Qué se le pidió?
- ¿Qué partes de la respuesta se aceptaron, rechazaron o modificaron?
- ¿Cómo se verificó el resultado?
- ¿Qué comprende ahora el estudiante que antes no comprendía?

La última pregunta permite relacionar el uso de la herramienta con el aprendizaje.

Evaluación de la comprensión

La fuente combina pruebas, entregas parciales, portafolios, registros y defensas orales.

Estas estrategias aportan evidencias sobre el proceso y la autoría, pero no garantizan por sí solas la ausencia de plagio ni demuestran que se haya evitado la pérdida de habilidades.

Experiencia Educativa Documentada y Evidencia Experimental Complementaria

Experiencia piloto descrita por Memari y Rudolph

Memari y Rudolph (2026, sección 9 y tabla 5) describen una asignatura optativa interdisciplinaria desarrollada durante dos semestres: otoño de 2025 y primavera de 2026.

Los estudiantes formaron equipos con funciones diferenciadas, utilizaron una plataforma comercial de orquestación de agentes sin código y abordaron problemas aportados por socios industriales regionales.

Los autores observaron que estudiantes de negocios podían trabajar con agentes y equipos de agentes sin haber cursado previamente programación.

Alcance de la evidencia del piloto

La experiencia es exploratoria, corresponde a un solo programa y no dispone de grupo de control.

Evidencia experimental complementaria: asistencia guiada en programación

Noraset et al. (2026a) realizaron un experimento controlado con 42 participantes en un curso breve de programación en Python.

El grupo que utilizó el asistente guiado presentó una mejora significativamente mayor entre la evaluación inicial y la final.

El estudio complementa el capítulo al documentar una experiencia evaluada en la que se buscó preservar la participación del estudiante durante la resolución de problemas.

La fe de erratas del estudio identifica a Mahidol University como institución donde se realizó la investigación y completa información sobre sus recursos y aprobación ética (Noraset et al., 2026b).

12.5 Síntesis Metodológica, Matriz de Decisión y Hoja de Ruta para Laboratorios de I+D

La aceleración en la validación empírica de hipótesis científicas no se logra adoptando una única herramienta tecnológica de forma aislada, sino mediante el establecimiento de un modelo de **Orquestación Híbrida**. Como demuestran las investigaciones de Marian et al. (2026), Khalfaoui et al. (2025) y Jauhari et al. (2026), los tres paradigmas emergentes —Low-Code empresarial determinista, generación basada en modelos UML e Inteligencia Artificial Generativa (*Vibe Coding*)— presentan fortalezas complementarias y limitaciones cruzadas.

El principio fundamental de la Orquestación Híbrida radica en desacoplar el prototipo científico en capas funcionales y asignar a cada capa el motor de construcción que maximice la eficiencia y minimice los riesgos de producción:

1. **Capa de Gobernanza y Procesos Documentales:** Asignada a plataformas Low-Code deterministas (ej. Microsoft Power Platform), aprovechando la autenticación centralizada mediante Entra ID, la persistencia en repositorios estructurados y el cumplimiento nativo de normas de auditoría documental (ISO 15489 e ISO 27001).
2. **Capa de Backend y Arquitectura de Datos Escalable:** Asignada a generadores dirigidos por modelos (ej. FSMicroGenerator), transformando diagramas de clases UML en arquitecturas de microservicios *full-stack* (stack MEAN) empaquetadas en contenedores Docker y monitoreadas en tiempo real.
3. **Capa de Exploración Inicial de UI/UX e Interacción en Lenguaje Natural:** Asignada al *Vibe Coding* respaldado por modelos de lenguaje (LLMs), permitiendo a los investigadores maquetar interfaces y probar conceptos de usuario en ciclos de horas, bajo la supervisión de herramientas de análisis estático de código (SAST).

Este enfoque consolida el paradigma de **reasignación de tareas** (*task reallocation*) teorizado por Memari y Rudolph (2026), garantizando que la automatización asuma la sobrecarga sintáctica mientras los investigadores ejercen el control arquitectónico y la validación de resultados.

Matriz Multidimensional Unificada de Selección Tecnológica

Para guiar la toma de decisiones técnicas en proyectos de I+D, se presenta la evaluación comparativa multidimensional consolidada a partir de los datos empíricos de las investigaciones analizadas:

Dimensión de Análisis	Low-Code Empresarial (Power Platform)	Generación UML (FSMicroGenerator)	Vibe Coding asistido por IA (LLMs)
Propósito Arquitectónico	Automatización de flujos documentales y aprobaciones.	Backend distribuido y microservicios escalables.	Maquetación rápida de UI/UX y exploración de conceptos.
Velocidad de Prototipado	Alta (Reducción de tiempo de ciclo del 65.6 %).	Extremadamente Alta (100 % funcionalidad en 3 horas).	Ultra Rápida (Aceleración de MVP entre 40 % y 60 %).
Control Arquitectónico	Moderado-Alto (Restringido por políticas del proveedor).	Total (Basado en patrones DDD y modelos UML).	Bajo-Moderado (Dependiente de la precisión del prompt).
Gobernanza y Auditoría	Nativa (Auditoría inmutable e ISO 15489).	Monitoreo expuesto en Prometheus y Grafana.	Requiere implementación externa de SAST / VIBE4M.
Esfuerzo Infraestructural	Mínimo (Nube SaaS/PaaS gestionada).	Moderado (Despliegue de contenedores Docker/GitLab).	Variable (Depende del entorno de ejecución del código).
Perfil de Usuario	Desarrollador Ciudadano / Investigador de campo.	Ingeniero de Software / Arquitecto de Datos.	Investigador multidisciplinario / Prototipador.

Análisis de Stakeholders (Matriz Poder-Interés) y Evaluación de TCO

La implementación de prototipos en organizaciones académicas y centros de investigación requiere gestionar las expectativas de los actores clave mediante una **Matriz de Poder e Interés**:

- **Gestionar Atentamente (Alto Poder, Alto Interés):** Investigadores Principales (PI) y Directores de Laboratorio. Exigen validez científica, rigor en el procesamiento de datos y cumplimiento de los plazos del proyecto.
- **Mantener Satisfechos (Alto Poder, Bajo Interés):** Comités de Ética, Oficiales de Seguridad de la Información (CISO) y Auditores Institucionales. Su prioridad es la protección de datos sensibles, el cumplimiento de la norma ISO 27001 y el control de accesos basado en roles (RBAC).
- **Mantener Informados (Bajo Poder, Alto Interés):** Estudiantes de posgrado, asistentes de investigación y desarrolladores ciudadanos. Requieren capacitación en herramientas de bajo código y guías transparentes sobre el uso ético de IA.
- **Monitorear (Bajo Poder, Bajo Interés):** Usuarios finales externos y evaluadores de pruebas piloto.

Evaluación del Costo Total de Propiedad (TCO)

El análisis económico revela diferencias estructurales entre los enfoques adoptados para el prototipado:

- **Enfoque Low-Code SaaS/PaaS (Power Platform):** Optimiza el TCO al aprovechar licencias educativas o corporativas preexistentes (Microsoft 365). La utilización de listas de SharePoint con columnas de búsqueda (*Lookup Columns*) permite modelar relaciones de datos sin incurrir en costos adicionales de bases de datos dedicadas como Microsoft Dataverse.
- **Enfoque Open-Source basado en Modelos (FSMicroGenerator):** Minimiza los costos recurrentes de licenciamiento al generar código ejecutable sobre tecnologías de código abierto (MongoDB, Express, Angular, Node.js, RabbitMQ). Sin embargo, requiere inversión inicial en infraestructura de cómputo para hospedar contenedores Docker y servidores de integración continua.

Algoritmo Formulado y Reglas de Decisión Arquitectónica

Para sistematizar la elección de herramientas en el laboratorio, se formula el siguiente algoritmo de decisión lógica:

```
INICIO: Evaluación del Proyecto de Investigación
|
+-- ?El proyecto requiere gestión de documentos, firmas o auditoría ISO?
|   |-- SÍ: Seleccionar Low-Code Empresarial (Power Apps + Automate)
|
+-- ?El proyecto requiere backend de alta concurrencia o DDD?
|   |-- SÍ: Diseñar UML -> Ejecutar FSMicroGenerator (Stack MEAN)
|
+-- ?El proyecto busca validar una interfaz de usuario o concepto exploratorio?
    |-- SÍ: Aplicar Vibe Coding con IA Generativa
    |
    +--> APLICAR CAPA DE ASEGURAMIENTO (Obligatorio):
         1. Registrar Prompts de generación.
         2. Ejecutar escaneo SAST (Trivy / SonarQube).
         3. Validar punto de Aprobación Humana (Human-in-the-Loop).
FIN
```

Mitigación del Triángulo de Riesgos y Capa de Aseguramiento

El prototipado acelerado expone a las instituciones a tres riesgos interconectados conocidos como el **Triángulo de Riesgos**:

1. **Acumulación de Deuda Técnica:** Código generado por IA sintética sin refactorización ni pruebas de integración.
2. **Atrofia Cognitiva (*Hollow Senior*):** Dependencia ciega del investigador en las salidas del modelo de lenguaje, perdiendo la capacidad de depurar errores complejos.
3. **Vulnerabilidades de Seguridad Ocultas:** Inyección de código propenso a fallos de ejecución o brechas de autenticación.

Para neutralizar estos riesgos, se exige la implementación transversal de la **Capa de Aseguramiento** (*Assurance Layer*) propuesta por Memari y Rudolph (2026) y reforzada por el marco de

gobernanza **VIBE4M** (Jauhari et al., 2026; Maes, 2025). Esta capa impone cinco mecanismos de auditoría obligatorios:

1. **Plantilla de Divulgación de IA:** Registro detallado donde el investigador documenta los modelos utilizados, los prompts formulados, el código generado aceptado/rechazado y las modificaciones manuales aplicadas.
2. **Pruebas Automatizadas Unitarias e Integradas:** Ejecución determinista de casos de prueba en canalizaciones CI/CD antes de permitir el despliegue del prototipo.
3. **Análisis Estático de Seguridad (SAST):** Escaneo automático de repositorios y contenedores para verificar la ausencia de credenciales expuestas y fallos de sintaxis.
4. **Punto de Aprobación Humana (*Human-in-the-Loop*):** Firma digital y revisión obligatoria por un investigador sénior antes de la puesta en producción.
5. **Defensa Oral y Justificación Arquitectónica:** Instancia de evaluación donde los creadores del prototipo deben justificar las decisiones de diseño y demostrar la comprensión completa del sistema generado.

Plan Maestro e Implementación en Laboratorios de I+D

La adopción institucional del prototipado acelerado se organiza en un Plan Maestro de cuatro fases proyectado a 12 meses:

- **Fase 1: Gobernanza y Capacitación (Meses 1-2):** Formulación de la política institucional de uso de IA y bajo código; instalación de herramientas de análisis estático (SAST) y capacitación de investigadores en modelado UML y diseño de prompts.
- **Fase 2: Piloto de Automatización Documental (Meses 3-6):** Despliegue de flujos deterministas L0-L2 mediante Power Platform para simplificar la gestión de protocolos y aprobaciones de laboratorio, midiendo reducciones en el tiempo de ciclo.
- **Fase 3: Escalado de Microservicios y Agentes (Meses 7-10):** Adopción de FSMicroGenerator para proyectos de backend distribuido e integración de agentes de IA L3-L4 con supervisión humana estricta.
- **Fase 4: Auditoría, Medición y Consolidación (Meses 11-12):** Evaluación de impacto mediante indicadores KPI (cobertura funcional, deuda técnica, costo operativo) y consolidación de un catálogo de componentes reutilizables para la universidad.

Notas bibliográficas e históricas

Antecedentes Teóricos Fundacionales:

La evolución del desarrollo acelerado de software se cimenta en la búsqueda progresiva de abstracción computacional. Sus raíces formales provienen del Desarrollo Rápido de Aplicaciones (RAD) de Martin (1991), el diseño dirigido por dominios (DDD) de Evans (2003) y los marcos de Ingeniería de Software Dirigida por Modelos (MDSE/MDD) sistematizados por Brambilla, Cabot y Wimmer (2017). Asimismo, la emergencia de la asistencia generativa por IA se apoya en los fundamentos de la arquitectura Transformer de Vaswani et al. (2017) y en la alineación mediante retroalimentación humana (RLHF) documentada por Ouyang et al. (2022).

Contribuciones de las Cinco Investigaciones Base:

El marco teórico y la evidencia empírica de este capítulo se fundamentan en cinco investigaciones científicas contemporáneas (2024–2026) que articulan el paradigma de prototipado científico acelerado:

- **Sido y Emon (2024) [Universidad de Aalborg]:** Establecen la distinción taxonómica formal entre plataformas No-Code y Low-Code, y analizan la interacción temprana con modelos generativos de IA en el ciclo de vida del software.
- **Marian, Neferu y Mitrea (2026) [Universidad POLITEHNICA de Bucarest]:** Demuestran empíricamente la viabilidad de plataformas Low-Code empresariales (Power Platform) para la gestión documental bajo normativas internacionales ISO 15489 e ISO 27001, aportando los datos de reducción del 65.6 % en tiempos de ciclo y 92 % de satisfacción usuaria.
- **Khalfaoui, Khalfaoui y Azmani (2025) [IEEE Access]:** Presentan el marco *FSMicroGenerator*, demostrando cuantitativamente la superioridad de la generación dirigida por modelos UML sobre el desarrollo manual (reducción del 92.4 % en esfuerzo y 100 % de cobertura funcional en 3 horas).
- **Jauhari, Fathullah, Permana y Nugraha (2026) [IJST]:** Proveen la Revisión Sistemática de Literatura (PRISMA sobre 61 estudios arbitrados) sobre *Vibe Coding*, conceptualizando el "cuello de botella de verificación", el síndrome del *Hollow Senior* y fundamentando el protocolo de gobernanza VIBE4M.
- **Memari y Rudolph (2026) [Utah Valley University / arXiv]:** Formulan la **Escalera de Capacidades** (niveles L0 a L4) y la **Capa de Aseguramiento** (*Assurance Layer*), definiendo el marco pedagógico y operativo de reasignación de tareas (*task reallocation*) frente a la automatización inteligente.

Lecturas Sugeridas:

1. **Brambilla, M., Cabot, J., & Wimmer, M. (2017):** *Model-Driven Software Engineering in Practice*. Obra de referencia indispensable para comprender la matemática y gramática de transformación de diagramas visuales a código ejecutable.
2. **Maes, S. H. (2025):** *The gotchas of AI coding and vibe coding*. Guía técnica esencial sobre gobernanza de código sintético y mitigación de deuda técnica en entornos de investigación.
3. **ISO 15489-1:2016:** *Information and documentation — Records management*. Estándar internacional para garantizar la trazabilidad, inmutabilidad y autenticidad en repositorios de datos científicos.

Resumen del capítulo 12

La aceleración del prototipado científico mediante software no depende de la adopción acrítica de una herramienta aislada, sino de la capacidad metodológica para implementar una orquestación híbrida que asigne a cada capa funcional el motor de desarrollo óptimo bajo estrictos estándares de gobernanza y aseguramiento de calidad.

01 **Orquestación Híbrida de Paradigmas:** Integración sinérgica por capas: plataformas Low-Code deterministas para flujos documentales y gobernanza; generadores dirigidos por modelos UML (*FSMicroGenerator*) para microservicios backend; y Vibe Coding con IA para interfaces exploratorias.

02 **Escalera de Capacidades y Reasignación Cognitiva:** Los niveles L0 a L4 permiten graduar la autonomía técnica de los sistemas, permitiendo que la automatización asuma la sobrecarga sintáctica repetitiva mientras el investigador reconcentra su esfuerzo en la arquitectura y la validación de hipótesis.

03 **Eficiencia Cuantitativa Comprobada:** Reducciones empíricamente medidas de más del 65.6 % en tiempos documentales (Power Platform) y del 92.4 % en desarrollo de microservicios con 100 % de cobertura funcional y 0 % de errores operativos bajo pruebas de estrés.

04 **Gobernanza, SAST y Capa de Aseguramiento:** La prevención de patologías de código sintético y riesgos OWASP exige análisis estático obligatorio en canalizaciones CI/CD, cumplimiento de directivas ISO 15489/27001 y supervisión obligatoria *Human-in-the-Loop*.

Síntesis: La combinación equilibrada entre bajo código, generación basada en modelos y modelos de lenguaje permite a las instituciones académicas acelerar drásticamente el ciclo de validación científica sin sacrificar el rigor metodológico, la seguridad ni la sostenibilidad técnica.

Conclusiones del capítulo 12

Conclusión principal

La convergencia entre plataformas Low-Code deterministas, generadores basados en modelos UML e Inteligencia Artificial Generativa representa un cambio de paradigma maduro para la investigación científica. La orquestación híbrida permite acortar sustancialmente los ciclos de desarrollo y llevar prototipos funcionales a validación empírica en cuestión de horas o días, siempre que esté respaldada por una Capa de Aseguramiento transversal que mitigue la acumulación de deuda técnica y garantice el cumplimiento normativo.

Aportes y límites

Aportes: Se sistematiza un marco de decisión tecnológica multidimensional, se demuestra la viabilidad de la automatización documental bajo normas ISO 15489/27001, y se articula la Escalera de Capacidades (L0–L4) con canalizaciones DevSecOps y análisis estático SAST. **Límites:** Las soluciones Low-Code propietarias conllevan riesgos de dependencia del proveedor (*vendor lock-in*) y costos de licenciamiento en escenarios de alta concurrencia; asimismo, el código generado por IA exige supervisión humana continua, siendo vulnerable a sesgos probabilísticos y fallos lógicos complejos que las herramientas automáticas no logran detectar de forma aislada.

Preguntas abiertas

¿De qué manera evolucionarán los marcos internacionales de auditoría y certificación de software a medida que los equipos de agentes autónomos de IA (Nivel L4) asuman la autogeneración, verificación y refactorización continua de prototipos en tiempo real sin intervención humana directa?

Referencias del Capítulo 12

Acemoğlu, D., Autor, D., Hazell, J., & Restrepo, P. (2022). Artificial intelligence and jobs. *Journal of Labor Economics*, 40(S1), S293–S340. https://doi.org/10.1086/718327

Borgonovi, F., Calvino, F., Criscuolo, C., et al. (2023). Emerging trends in AI skill demand across 14 OECD countries. *OECD Artificial Intelligence Papers*. https://doi.org/10.1787/7c691b9a-en

Brambilla, M., Cabot, J., & Wimmer, M. (2017). *Model-Driven Software Engineering in Practice* (2nd ed.). Morgan & Claypool. https://doi.org/10.2200/S00751ED2V01Y201701SWE004

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley. ISBN: 978-0-321-12521-7.

Fowler, M. (2014). *Microservices Architecture*. https://martinfowler.com/articles/microservices.html

Fu, X., Bultan, T., & Su, J. (2002). Formal verification of e-services and workflows. En *Web Services, E-Business, and the Semantic Web* (pp. 188–202). Springer. https://doi.org/10.1007/3-540-36189-8_15

ISO 15489-1:2016. (2016). *Information and documentation — Records management — Part 1: Concepts and principles*. International Organization for Standardization. https://www.iso.org/standard/62542.html

ISO/IEC 27001:2022. (2022). *Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. International Organization for Standardization. https://www.iso.org/standard/27001

Jauhari, A. F., Fathullah, F., Permana, I. A., & Nugraha, R. (2026). A systematic analysis of AI-assisted vibe coding in software development: Opportunities, challenges, and risks. *International Journal Science and Technology*, 5(2), 261–273. https://doi.org/10.56127/ijst.v5i2.3000

Khalfaoui, S., Khalfaoui, H., & Azmani, A. (2025). Microservices-driven automation in full-stack development: Bridging efficiency and innovation with FSMicroGenerator. *IEEE Access*, 13, 125131–125156. https://doi.org/10.1109/access.2025.3589285

Maes, S. H. (2025). *The gotchas of AI coding and vibe coding: Support and maintenance in agentic software engineering*. Zenodo. https://doi.org/10.5281/zenodo.14889210

Manca, F. (2023). Six questions about the demand for artificial intelligence skills in labour markets. *OECD Social, Employment and Migration Working Papers*, 286. https://doi.org/10.1787/ac1bebf0-en

Marian, C. V., Neferu, M., & Mitrea, D. A. (2026). Design and evaluation of a low-code/no-code document management and approval system. *Information*, 17(1), 46. https://doi.org/10.3390/info17010046

Martin, J. (1991). *Rapid Application Development*. Macmillan Publishing Company. ISBN: 978-0-02-376775-3.

Memari, M., & Rudolph, G. (2026). The Capability Ladder: A curriculum-modernization framework for workforce readiness in the AI era. *arXiv preprint* arXiv:2608.07779. https://doi.org/10.48550/arXiv.2608.07779

Meske, C., Hermanns, T., Weiden, E., Loser, K., & Berger, T. (2025). Vibe coding as a reconfiguration of intent mediation in software development. *IEEE Access*, 13, 45120–45135. https://doi.org/10.1109/ACCESS.2025.3519802

Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. OŔeilly Media. ISBN: 978-1-491-95035-7.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 27730–27744. https://doi.org/10.48550/arXiv.2203.02155

Prebreza, B., Gotseva, D., & Nakov, P. (2021). A study of documents management system based on web, case study: University. *2021 29th National Conference with International Participation (TELECOM)*, 85–89. https://doi.org/10.1109/telecom53156.2021.9659663

Royce, W. W. (1970). Managing the development of large software systems: Concepts and techniques. *Proceedings of IEEE WESCON*, 26, 1–9. IEEE Computer Society.

Sartori, C. C. (2025). Architectures of error: A philosophical inquiry into AI and human code generation. *SSRN Electronic Journal*. https://doi.org/10.2139/ssrn.5265751

Shi, Z., Dong, J., & Gan, Y. (2025). Democratizing digital transformation: A multisector study of low-code adoption patterns. *Applied Sciences*, 15(12), 6481. https://doi.org/10.3390/app15126481

Sido, N., & Emon, E. A. (2024). *Low/No code development and generative AI* [Masterś thesis, Aalborg University]. Aalborg University Digital Repository. https://projekter.aau.dk/projekter/da/studentprojects/lowno-code-development-and-generative-ai

van Hurne, M. H. K. (2025). A technical debt-aware prompting framework for sustainable vibe coding. *TechRxiv*. https://doi.org/10.36227/techrxiv.175459417.76916566/v1

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008. https://doi.org/10.48550/arXiv.1706.03762

Zhao, S., Wang, D., Zhang, K., Chen, Y., & Liu, H. (2025). Is vibe coding safe? Benchmarking security vulnerabilities of agent-generated code in real-world scenarios. *arXiv preprint* arXiv:2512.03262. https://doi.org/10.48550/arXiv.2512.03262

Sobre Esta Obra

Sinopsis:

Inteligencia Artificial Aplicada: Fundamentos, Métodos y Validación Científica constituye una obra académica de referencia avanzada que sintetiza los avances, metodologías y debates contemporáneos en torno a la inteligencia artificial, la automatización del desarrollo de software y la gobernanza de sistemas autónomos. Producida por la Facultad de Ciencias Matemáticas de la Universidad Nacional Mayor de San Marcos (UNMSM) bajo una estricta maqueta editorial institucional y con citación estandarizada en Normas APA 7ł edición, esta obra examina desde los fundamentos de la abstracción visual y la generación de microservicios desde modelos UML, hasta las implicancias éticas del código sintético (*Vibe Coding*), la mitigación del triángulo de riesgos y la supervisión humana responsable en la era de los agentes de IA.

Diseñado tanto para estudiantes universitarios como para docentes, científicos e ingenieros en ejercicio, este volumen proporciona un mapa de ruta replicable para liderar la transformación digital de la investigación con rigor metodológico, velocidad experimental y estricto compromiso ético.

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América · Fundada en 1551

Facultad de Ciencias Matemáticas · Lima, Perú